EC528 Demo-2

# Rust-based Openstack-Keystone:Token Exchange

Dominic Murphy, Zebang Li

Thinh Nguyen, Kevin Wang




openstack.

# Problem and Motivation for RFC 8693 (Token Exchange)

Keystone's token verifier assumes every subject token is a Keystone token, including an audience specific to Keystone. That means it **can't safely accept tokens from other trusted issuers** as part of token exchange.

We're introducing shared validation profiles so **each kind of subject token can have the right issuer, audience, required claims, and time limits**, while preserving existing Keystone-token behavior from the Python version of Keystone.

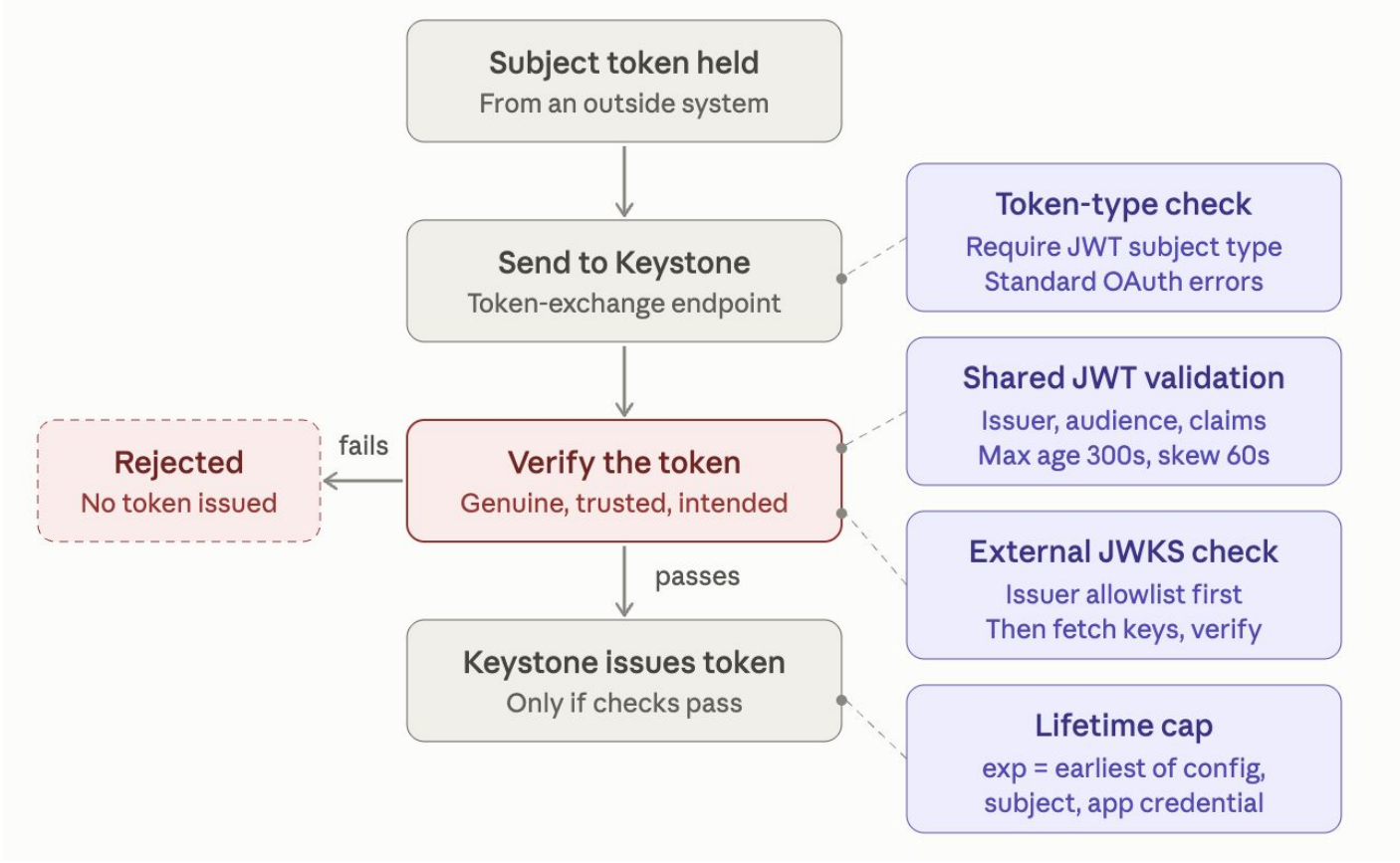
**The end result is a modernized IAM ecosystem**

# Milestone Review and Progress

| Demo | Date | Milestone | How we will demonstrate it |
|---|---|---|---|
| Demo 2 | 10/21 | Endpoint Dispatch & Validation Profile | Unit tests confirming /token correctly routes grant_type=token-exchange, and verify.rs successfully accepts dynamic per-subject validation profiles instead of hardcoded constants |
| Demo 2 | 10/21 | Signature Verification | Integration test demonstrating successful fetching of an external JWKS and verification of a mock subject_token signature. |

- All Demo-2 deliverables have been finished and system/unit tested
- PRs remain under review and are pending merge to the OpenStack repo.
- More on our progress next…

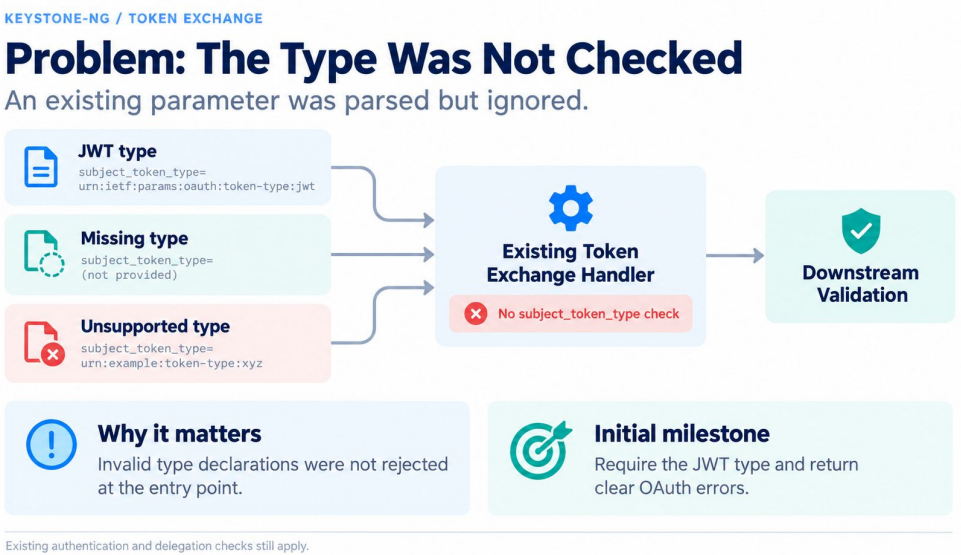
# Our PRs Visualized



# Problem: Missing Token-Type Validation

- The token-exchange endpoint accepted `subject_token_type` but did not validate it.
- Missing or unsupported types could reach downstream processing.
- Goal: enforce an explicit type requirement and provide clear OAuth errors.

**My scope:** Request-type validation within the existing exchange flow.

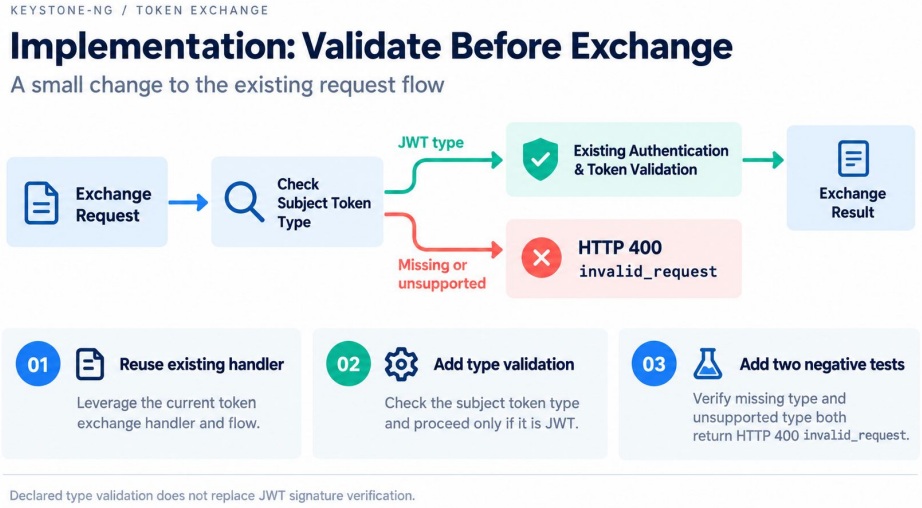


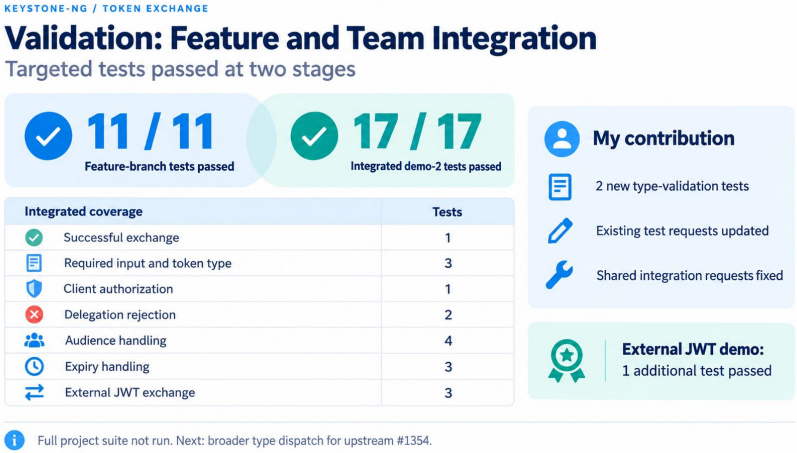
# Implementation: Validate Before Exchange

- Reused the existing token-exchange handler.
- Required the JWT subject-token type as an initial milestone.
- Returned HTTP 400 with `invalid_request` for missing or unsupported types.
- Preserved existing authentication and delegation checks.

**Flow:** Request → Type check → Existing validation → Exchange result

**Boundary:** Type checking does not replace JWT signature verification.



# PR #1459 - Token Exchange Can Refresh Lifetime Indefinitely

Two different types of ‘tokens’: Subject Token and Application Credential

- Subject tokens associated with user
- Application credential associated with service (Nova, Cinder)

Previous code: `Token exp = now + access_token_lifetime_minutes * 60`

- Does not account for existing token expiration and instead uses a constant refresh valve
- RFC 8693: Tokens may not expire later than the user’s overall authorization

Malicious attacker (if seized your subject token) can infinitely regenerate your token

- Conduct actions on your behalf
- Access sensitive data or services

**However**; Openstack does generate permanent (no expiry) application credentials, which should be allowed.

# PR #1459 - Token Exchange Can Refresh Lifetime Indefinitely

Solution is straightforward:

- Cap the total lifetime of a token to the configured token lifetime.
  - IE: After refresh you cannot exceed more than 15 minutes (etc)
  - We still refresh in intervals. If a token has been alive for 1 minute, we will refresh for another 1 minute not 14 minutes.
- Applicational credentials; If permanent we will either:
  - Apply an expiry date based on a default configuration
  - Pass it through as is, depending on the type
  - Or if it has an expiry: Apply normal refresh based on above rules
- Edge cases for already expired, will expiry is too soon, or the expiry is somehow before the issue time.

# Speed Context

| Term | Plain-English meaning | Everyday analogy |
|---|---|---|
| Keystone | OpenStack's identity service. It decides who you are and what you may do in the cloud. | The building's security desk |
| Identity provider (IdP) | A service that logs people in and vouches for them. | The DMV that issues driver's licenses |
| Federation | Keystone agreeing to accept logins vouched for by an outside IdP. | Accepting a partner company's ID badge |
| Token | A digital proof of identity or permission that is handed to a user or app. | A wristband at a festival |
| Subject token | The token someone presents to say 'this is who I am'. | The ID you show at the door |

Speed Context

| JWT part | What it contains | Passport analogy |
|---|---|---|
| Header | Which signing method was used (the algorithm) | The type of passport and security features |
| Payload | The claims: facts about the token, such as who issued it, who it is for, and when it expires | The data page: name, nationality, expiry date |
| Signature | A cryptographic seal made with the issuer's private key. Anyone with the matching public key can check it. | The hologram seal that proves it is not forged |

# Speed Context

| Claim | What it says | Why it is checked |
|---|---|---|
| iss (issuer) | Who created this token | Only tokens from organizations we have chosen to trust should be accepted |
| aud (audience) | Who the token is meant for | A token made for Service A should not work at Service B |
| sub (subject) | Who the token is about | Identifies the person or app being vouched for |
| exp / iat (expiry / issued-at) | When the token stops being valid and when it was created | Old or future-dated tokens are rejected |
| jti (token ID) | A unique ID for this one token | Lets a system recognize a token being reused (a replay) |

# PR #1458 — Add external JWT verification

PROBLEMS IDENTIFIED:

- **Anyone can write anything in a token's issuer field**

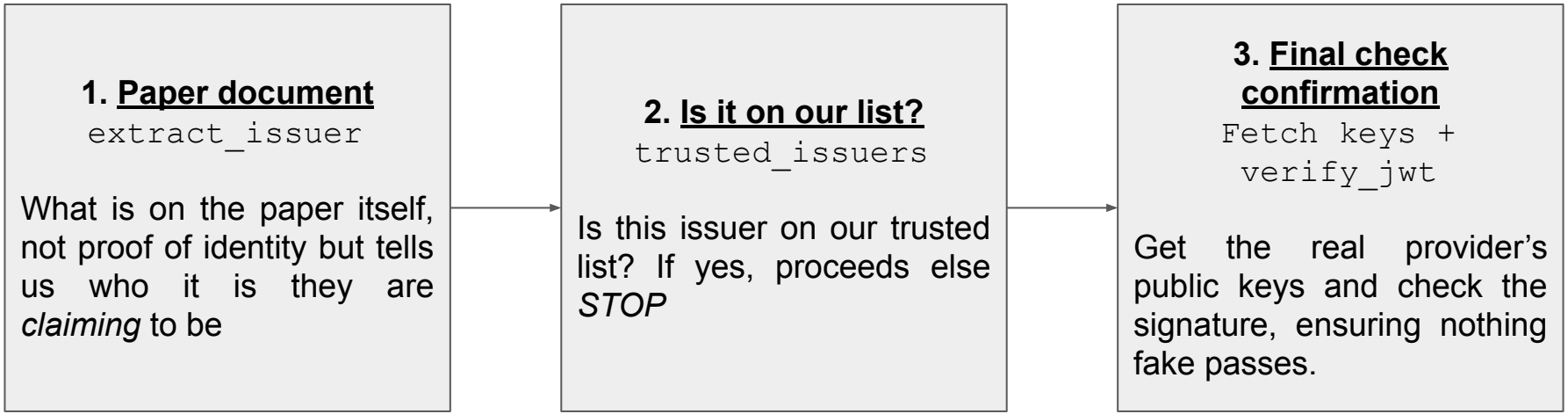
  -> Until the signature is checked, 'iss' is only a claim, not a fact
- **Looking up unknown issuers is dangerous**

  -> Fetching keys for any issuer an attacker names could make Keystone send unwanted web requests (an SSRF-style risk)

- **The existing verifier skips the audience check if no audience is supplied**

  -> A genuine token meant for another service could be accepted

# How do we verify it? - Passport analogy



# Results

| Scenario tested | What it proves, in plain English |
|---|---|
| Valid token | The good case still works |
| Malformed token; invalid iss | Garbage input is rejected cleanly |
| Untrusted issuer makes no network requests | Attackers cannot make Keystone fetch from arbitrary places |
| Wrong or missing audience | A token for another service is refused |
| Expired token | Old tokens stop working |
| Tampered payload or signature | Any edit to the token is detected |
| Unknown or incorrect signing key | A token signed by the wrong key is refused |
| Discovery issuer mismatch | Discovery data cannot quietly substitute a different issuer |
| Symmetric algorithms | Shared-secret signing is handled safely |
| Static JWKS preferred; JWKS fetch failure | Configured keys win, and failures are handled |
| Issuer text kept out of logs | Attacker-controlled text is not echoed |

# PR #1560 — Add shared OAuth2 subject-token validation

DIFFERENT TOKENS, DIFFERENT RULES:

- **Keystone's own tokens** are checked for use, revocation (cancelled tokens) and delegation (authority passed to someone else)

- •**Outside tokens** need stricter rules: exact audience, required claims, short lifetimes

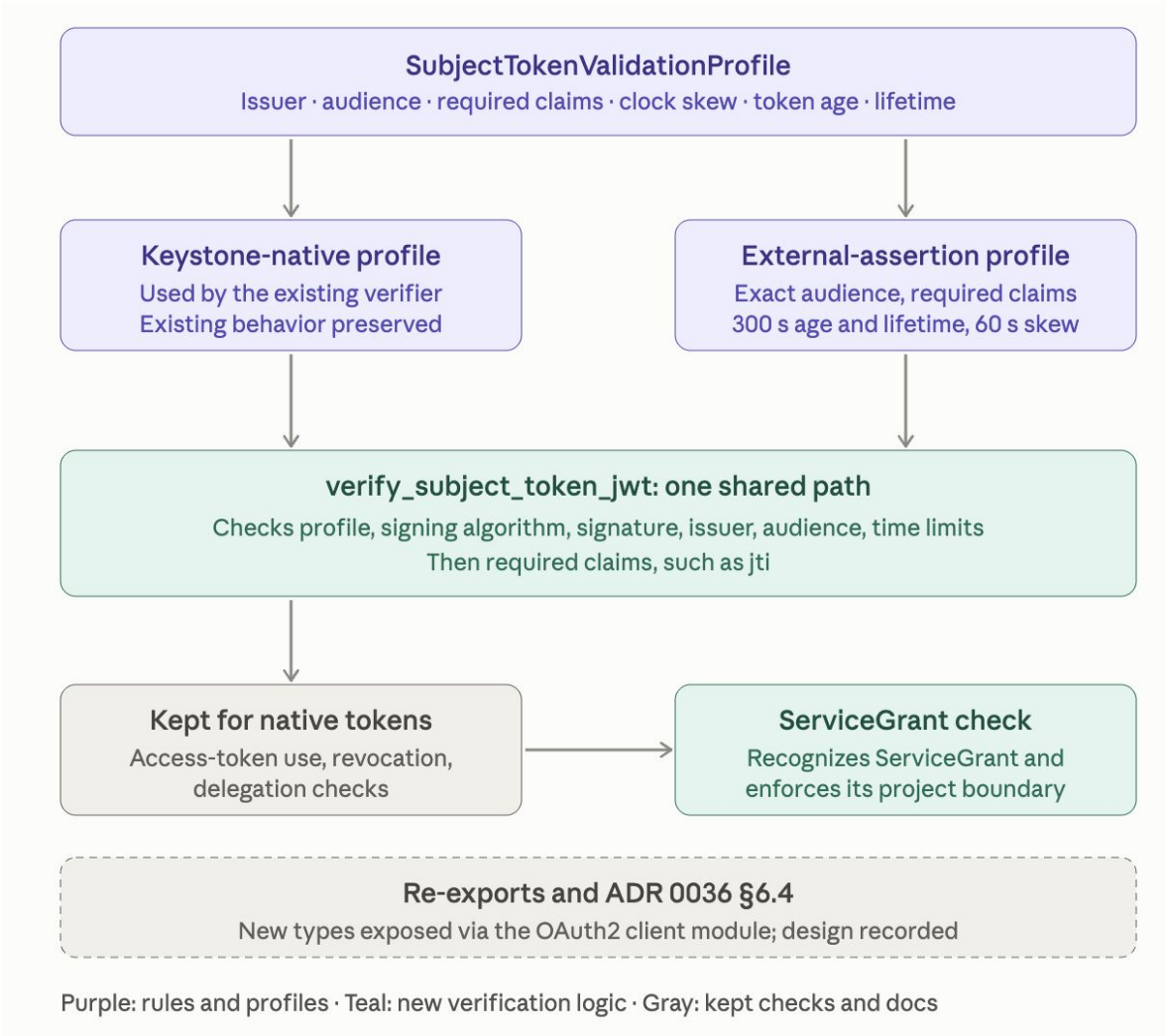
  -> Those rules were not bundled together or selectable by token type

- **Custom required claims such as jti** are not enforced by the JWT library's standard setting

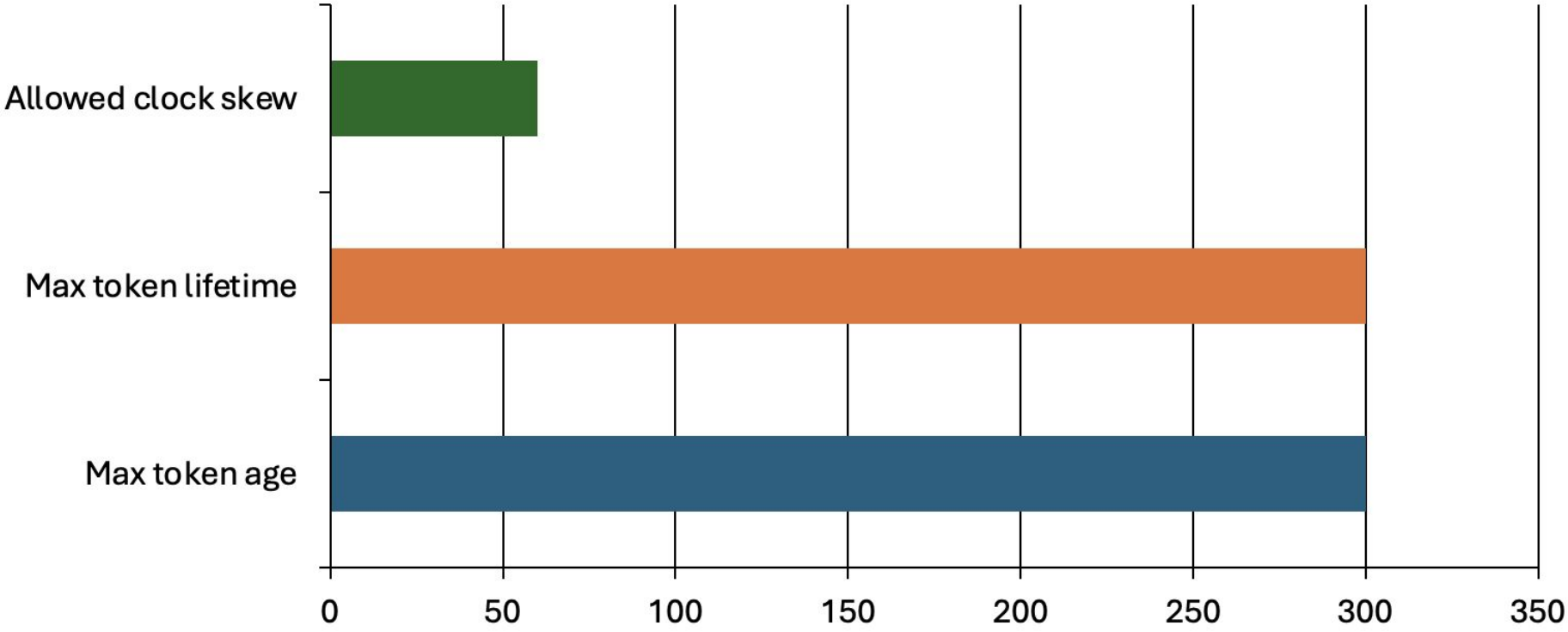
**Constraint:** do not weaken today's Keystone checks while adding the new rules

# The Fix

- `SubjectTokenValidationProfile` at the top defines what “acceptable” means. External-assertions have a strict 300 s age and lifetime and 60 s skew.
- Both profiles feed `verify_subject_token_jwt`, the single verification path.
- The native profile continues into the checks it keeps (access-token use, revocation, delegation), and those flow into the new `ServiceGrant` check, which enforces the project boundary.



# The External-Assertion Profile's Time Limits (seconds)



# Results

**46 targeted tests passed**

- Shared-token profiles
  - Checks that tokens satisfy the selected profile's issuer, audience, required-claim, signature, and time limits.
- OIDC token verification
  - Checks that native access tokens meet expected audience, issuer, expiry, revocation, signing-algorithm, and delegation rules.
- OIDC access token verification
  - Checks that OIDC access tokens are valid, unrevoked, and include the openid scope, while rejecting unsuitable OpenStack tokens.
- ID-token hint verification
  - Checks that logout ID-token hints are correctly signed and typed, including acceptance of expired ID tokens and rejection of invalid tokens.
- Revocation and introspection
  - Checks that revocation is limited to the owning client and that introspection accurately reports token validity and revocation.
- Multi-issuer verification
  - Checks that tokens from multiple trusted issuers are validated with the correct issuer-specific keys.

# Full Prototype Demo Statistics

- 4 Pull Requests to OpenStack’s Keystone Repository with key features for compliant token exchange
- 64 custom unit tests
- >1000 formatting, linting, security, collaborator standard tests passed
- 1495 new lines of code

Thank you!